SYSTÈME DE GESTION DE PHARMACIE

Rapport de projet

Réalisé par
Youssef Jbeniani

Application développée en Java avec une base de données SQLite et JDBC

1. Introduction

Ce projet consiste à concevoir et à réaliser une application de gestion d'une pharmacie. L'objectif est de centraliser les opérations courantes, de faciliter le suivi des médicaments et des ventes, et de garder une trace des mouvements de stock.

1.1 Objectifs fonctionnels

- L'authentification des utilisateurs selon leur rôle : pharmacien ou gestionnaire.
- La gestion des médicaments : ajout, suppression, modification du prix et mise à jour du stock.
- L'enregistrement et l'annulation des ventes, avec actualisation du stock.
- La consultation des ventes par médicament, par client et par période.
- La gestion des commandes de réapprovisionnement et la consultation des commandes.
- Le repérage des médicaments dont le stock est inférieur à un seuil critique.
- La conservation de l'historique des entrées et des sorties de stock dans la table StockHistorique.

1.2 Technologies utilisées

L'application est développée en Java. Les données sont stockées dans une base SQLite, à laquelle l'application accède avec JDBC. Le rapport présente le modèle de données, les principales classes, la structure de la base et quelques opérations exécutées dans l'application.

2. Modèle conceptuel des données

Le modèle s'appuie sur sept entités principales. Chaque entité possède un identifiant et les attributs nécessaires à la gestion des utilisateurs, des clients, des médicaments, des ventes, des commandes et des mouvements de stock.

2.1 Entités et attributs

Entité	Attributs principaux
PHARMACIEN	id_pharmacien, nom, prenom, login (unique), pwd
GESTIONNAIRE	id_gestionnaire, nom, prenom, login (unique), pwd
CLIENT	id_client, nom, prenom, email, adresse
MEDICAMENT	id_medicament, nom, dosage, stock, prix_unitaire
VENTE	id_vente, date, quantite
COMMANDE	id_commande, date, quantite
STOCK_HISTORIQUE	id_stock_hist, date, quantite (variation positive ou négative)

2.2 Associations et cardinalités

Les cardinalités décrivent les liens entre les entités à partir des clés étrangères présentées dans le schéma :

- Un pharmacien peut réaliser zéro à plusieurs ventes ; chaque vente est enregistrée par un seul pharmacien.
- Un client peut effectuer zéro à plusieurs ventes ; chaque vente concerne un seul client.
- Un médicament peut apparaître dans zéro à plusieurs ventes ; chaque vente concerne un seul médicament.

- Un gestionnaire peut créer zéro à plusieurs commandes ; chaque commande est créée par un seul gestionnaire.
- Un médicament peut apparaître dans zéro à plusieurs commandes ; chaque commande concerne un seul médicament.
- Un médicament peut posséder zéro à plusieurs lignes d’historique ; chaque ligne concerne un seul médicament.

3. Conception UML

Le diagramme de classes présente les principales classes du système et leurs relations. Le diagramme de cas d'utilisation résume les actions disponibles pour les deux rôles : pharmacien et gestionnaire.

3.1 Diagramme de classes

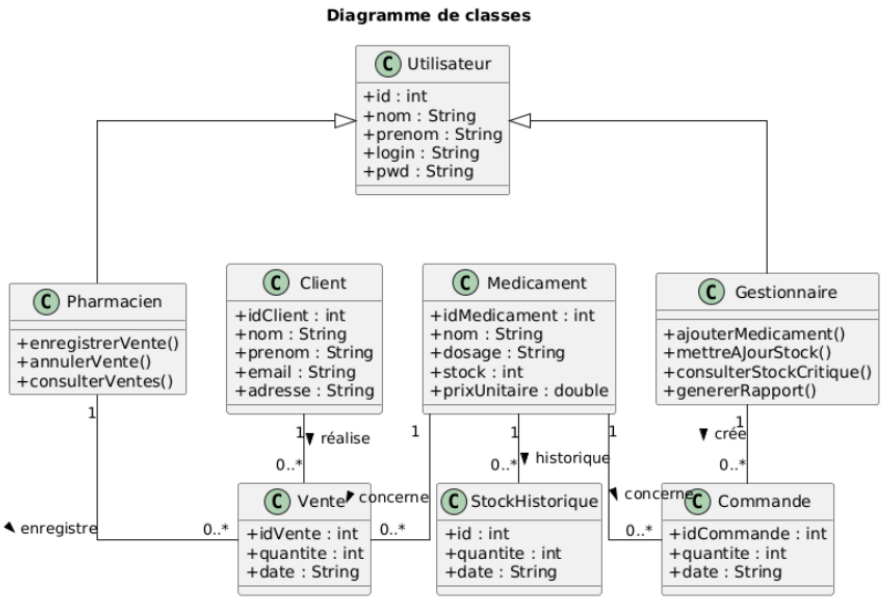
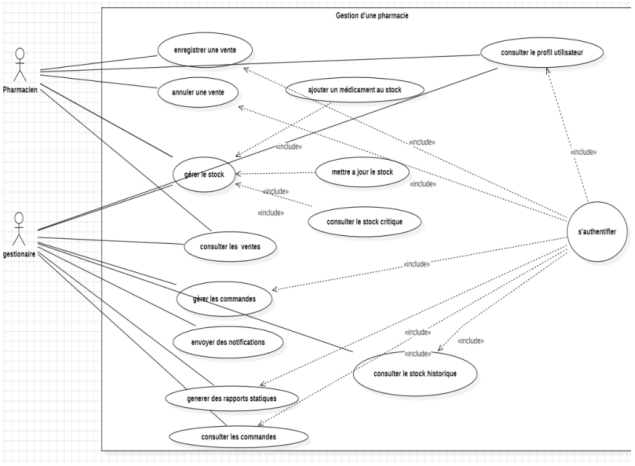


Diagramme de cas/utilisation:

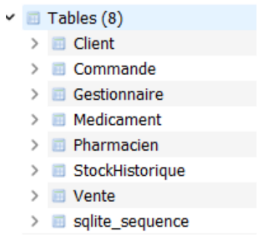
Figure 1. Diagramme de classes fourni dans le projet.

3.2 Diagramme de cas d’utilisation



Base de donnees :

Les tables:



Exemples de contenue des tables :

Figure 2. Diagramme de cas d’utilisation et aperçu de la structure de la base.

4. Base de données

La base de données SQLite regroupe les tables nécessaires au fonctionnement de l'application. Le schéma fourni montre les tables Client, Commande, Gestionnaire, Medicament, Pharmacien, StockHistorique et Vente, ainsi que la table interne sqlite_sequence.

Les opérations réalisées dans l'application s'appuient sur ces données pour enregistrer les ventes, mettre à jour les quantités disponibles et conserver l'historique des mouvements de stock.

5. Organisation du code

Le projet est organisé autour de classes représentant les objets métier et la session de l'utilisateur. L'architecture mentionnée dans le projet distingue les couches DAO, Service et App afin de séparer l'accès aux données, la logique applicative et le lancement de l'application.

5.1 Classes présentées

- Client
- Gestionnaire
- Pharmacien
- Commande
- Medicament
- Session
- Vente

Les captures suivantes reprennent les éléments de code présents dans le document initial.

6. Captures du code source

Les classes du code :

Client:

```java
public class Client {

    private int idClient;
    private String nom;
    private String prenom;
    private String email;
    private String adresse;


    public Client(int idClient, String nom, String prenom, String email, String adresse) {
        this.idClient = idClient;
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
        this.adresse = adresse;
    }
```

Gestionnaire:

```java
public class Gestionnaire {

    private int idGestionnaire;
    private String nom;
    private String prenom;
    private String login;
    private String pwd;

    public Gestionnaire(int idGestionnaire, String nom, String prenom, String login, String pwd) {
        this.idGestionnaire = idGestionnaire;
        this.nom = nom;
        this.prenom = prenom;
        this.login = login;
        this.pwd = pwd;
    }
```

Classe Client

Pharmacien:

```java
public class Pharmacien {

    private int idPharmacien;
    private String nom;
    private String prenom;
    private String login;
    private String pwd;



    public Pharmacien(int idPharmacien, String nom, String prenom, String login, String pwd) {
        this.idPharmacien = idPharmacien;
        this.nom = nom;
        this.prenom = prenom;
        this.login = login;
        this.pwd = pwd;
    }
```

Commande:

```java
public class Commande {

    private int idCommande;
    private int idGestionnaire;
    private int idMedicament;
    private int quantite;
    private String date; // ISO string

    public Commande(int idCommande, int idGestionnaire, int idMedicament, int quantite, String date) {
        this.idCommande = idCommande;
        this.idGestionnaire = idGestionnaire;
        this.idMedicament = idMedicament;
        this.quantite = quantite;
        this.date = date;
    }
```

Medicament:

```java
public class Medicament {

    private int idMedicament;
    private String nom;
    private String dosage;
    private int stock;
    private double prixUnitaire;

    public Medicament(int idMedicament, String nom, String dosage, int stock, double prixUnitaire) {
        this.idMedicament = idMedicament;
        this.nom = nom;
        this.dosage = dosage;
        this.stock = stock;
        this.prixUnitaire = prixUnitaire;
    }
```

Session:

Classes Gestionnaire et Pharmacien

```java
public class Session {

    private String role;

    private int userId;
    private String nom;
    private String prenom;

    public Session(String role, int userId, String nom, String prenom) {
        this.role = role;
        this.userId = userId;
        this.nom = nom;
        this.prenom = prenom;
    }
```

Vente:

```java
public class Vente {

    private int idVente;
    private int idPharmacien;
    private int idClient;
    private int idMedicament;
    private int quantite;
    private String date;


    public Vente(int idVente, int idPharmacien, int idClient, int idMedicament, int quantite, String date) {
        this.idVente = idVente;
        this.idPharmacien = idPharmacien;
        this.idClient = idClient;
        this.idMedicament = idMedicament;
        this.quantite = quantite;
        this.date = date;
```

L'application:

Classes Commande et Medicament



```java
public class App {

    // Scanner unique pour toute l'application (lecture console).

    private static final Scanner sc = new Scanner(System.in);

    // Méthode principale : boucle infinie de login -> menu -> logout.

    public static void main(String[] args) {
        System.out.println("=== PHARMACIE APP (JDBC + SQLite) ===");

        // Service d'authentification (appelle AuthDao -> SQLite)
        AuthService authService = new AuthService();

        while (true) {
            System.out.println();
            System.out.println("----- LOGIN -----");

            // Saisie login / mot de passe
            String login = readLine("Login: ");
            String pwd = readLine("Password: ");

            // Tentative de connexion
            Session session;
            try {
                session = authService.login(login, pwd);
            } catch (Exception e) {
                // Erreur technique (ex: driver, base introuvable, SQL error)
                System.out.println(" Erreur login: " + e.getMessage());
                continue;
            }

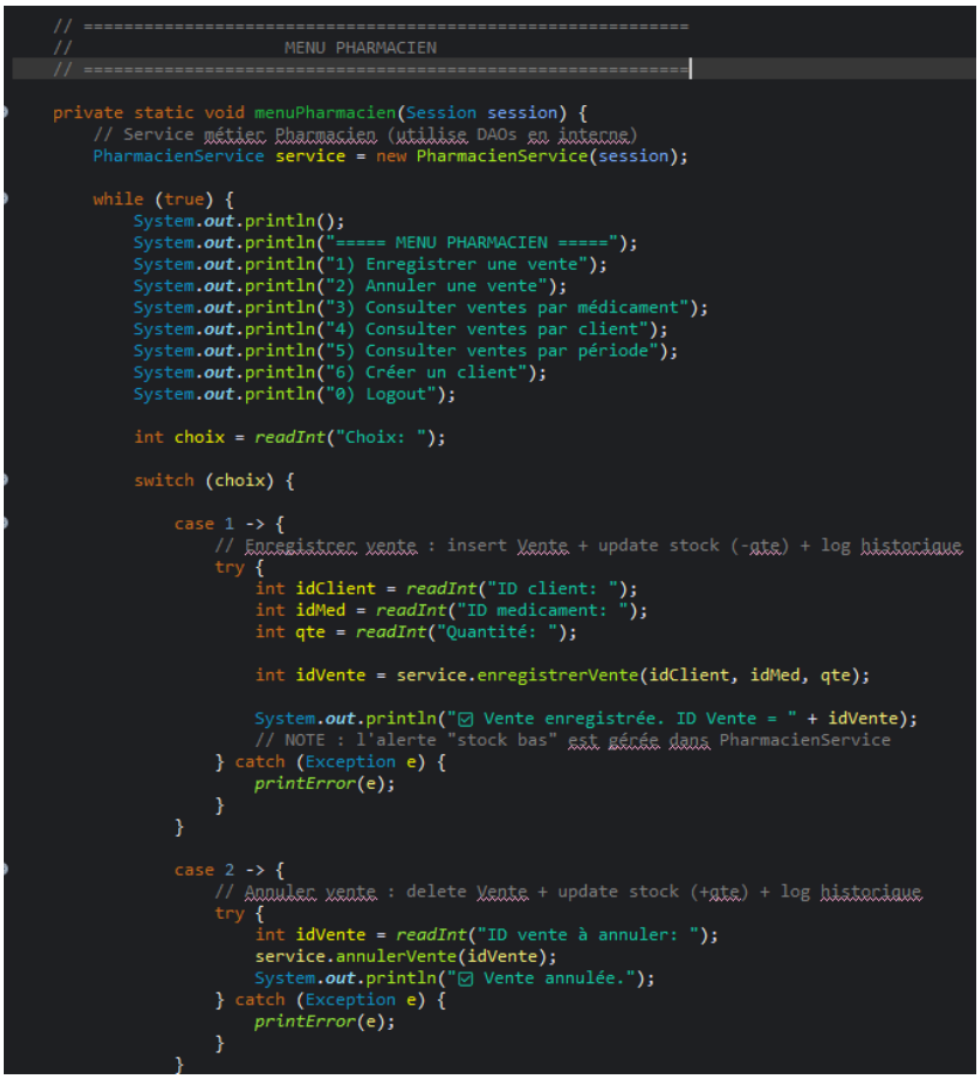
            // Si session == null => login/pwd incorrects
            if (session == null) {
                System.out.println(" Login incorrect.");
                continue;
            }

            System.out.println(" Connecté: " + session.getRole() + " | "
                    + session.getPrenom() + " " + session.getNom());

            // Redirection selon le rôle
            String role = session.getRole() == null ? "" : session.getRole().toLowerCase();
            if (role.contains("pharmac")) {
                menuPharmacien(session);
            } else if (role.contains("gestion")) {
                menuGestionnaire(session);
            } else {
                System.out.println("⚠ Rôle inconnu: " + session.getRole());
            }

            // Retour ici après logout
            System.out.println(" Déconnecté.");
        }
    }
```

Classe Session



```
    // ============================================================
    //                      MENU PHARMACIEN
    // ============================================================

    private static void menuPharmacien(Session session) {
        // Service métier Pharmacien (utilise DAOs en interne)
        PharmacienService service = new PharmacienService(session);

        while (true) {
            System.out.println();
            System.out.println("===== MENU PHARMACIEN =====");
            System.out.println("1) Enregistrer une vente");
            System.out.println("2) Annuler une vente");
            System.out.println("3) Consulter ventes par médicament");
            System.out.println("4) Consulter ventes par client");
            System.out.println("5) Consulter ventes par période");
            System.out.println("6) Créer un client");
            System.out.println("0) Logout");

            int choix = readInt("Choix: ");

            switch (choix) {

                case 1 -> {
                    // Enregistrer vente : insert Vente + update stock (-qte) + log historique
                    try {
                        int idClient = readInt("ID client: ");
                        int idMed = readInt("ID medicament: ");
                        int qte = readInt("Quantité: ");

                        int idVente = service.enregistrerVente(idClient, idMed, qte);

                        System.out.println("☑ Vente enregistrée. ID Vente = " + idVente);
                        // NOTE : l'alerte "stock bas" est générée dans PharmacienService
                    } catch (Exception e) {
                        printError(e);
                    }
                }

                case 2 -> {
                    // Annuler vente : delete Vente + update stock (+qte) + log historique
                    try {
                        int idVente = readInt("ID vente à annuler: ");
                        service.annulerVente(idVente);
                        System.out.println("☑ Vente annulée.");
                    } catch (Exception e) {
                        printError(e);
                    }
                }
```

Classe Vente



```
case 3 -> {
    // Consulter ventes par médicament
    try {
        int idMed = readInt("ID medicament: ");
        List<VenteDao.Vente> ventes = service.ventesParMedicament(idMed);

        if (ventes.isEmpty()) {
            System.out.println("(Aucune vente)");
        } else {
            for (VenteDao.Vente v : ventes) {
                System.out.println("Vente#" + v.idVente()
                        + " | client=" + v.idClient()
                        + " | qte=" + v.quantite()
                        + " | date=" + v.date());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}

case 4 -> {
    // Consulter ventes par client
    try {
        int idClient = readInt("ID client: ");
        List<VenteDao.Vente> ventes = service.ventesParClient(idClient);

        if (ventes.isEmpty()) {
            System.out.println("(Aucune vente)");
        } else {
            for (VenteDao.Vente v : ventes) {
                System.out.println("Vente#" + v.idVente()
                        + " | med=" + v.idMedicament()
                        + " | qte=" + v.quantite()
                        + " | date=" + v.date());
            }
        }
    } catch (Exception e) {
        printError(e);
    }
}
```

Capture complémentaire du code

7. Présentation de l’application

Les captures de l’application illustrent plusieurs opérations courantes liées à la gestion des ventes, à la gestion des médicaments et au réapprovisionnement.

7.1 Enregistrement et annulation d’une vente

Execution (exemples):

Enregistrer une vente (Pharmacien) :

```
----- LOGIN -----
Login: pharma
Password: pharma123
☑ Connecté: PHARMACIEN | Pharma Doe

===== MENU PHARMACIEN =====
1) Enregistrer une vente
2) Annuler une vente
3) Consulter ventes par médicament
4) Consulter ventes par client
5) Consulter ventes par période
6) Créer un client
0) Logout
Choix: 1
ID client: 1
ID medicament: 2
Quantité: 1
☑ Vente enregistrée. ID Vente = 2
```

Annulation d’une vente (Pharmacien) :

```
----- LOGIN -----
Login: pharma
Password: pharma123
☑ Connecté: PHARMACIEN | Pharma Doe

===== MENU PHARMACIEN =====
1) Enregistrer une vente
2) Annuler une vente
3) Consulter ventes par médicament
4) Consulter ventes par client
5) Consulter ventes par période
6) Créer un client
0) Logout
Choix: 2
ID vente à annuler: 1
☑ Vente annulée.
```

Exécution : enregistrement d’une vente par le pharmacien

Ajouter médécament (Gestionnaire):

```
----- LOGIN -----
Login: gest
Password: gest123
☑ Connecté: GESTIONNAIRE | Gestion Doe

===== MENU GESTIONNAIRE =====
1) Ajouter médicament
2) Mettre à jour stock (valeur exacte)
3) Afficher stock critique
4) Créer commande (réapprovisionnement)
5) Consulter commandes
6) Historique stock d'un médicament
7) Lister tous les médicaments
8) Modifier prix d'un médicament
0) Logout
Choix: 1
Nom: Motilium
Dosage: 10
Stock initial: 14
Prix unitaire: 15
☑ Médicament ajouté. ID = 7
```

Modifier prix d'un mediament (Gestionnaire) :

```
----- LOGIN -----
Login: gest
Password: gest123
☑ Connecté: GESTIONNAIRE | Gestion Doe

===== MENU GESTIONNAIRE =====
1) Ajouter médicament
2) Mettre à jour stock (valeur exacte)
3) Afficher stock critique
4) Créer commande (réapprovisionnement)
5) Consulter commandes
6) Historique stock d'un médicament
7) Lister tous les médicaments
8) Modifier prix d'un médicament
0) Logout
Choix: 8
ID medicament: 7
Nouveau prix: 13
☑ Prix modifié.
```

Exécution : annulation d'une vente par le pharmacien

Conclusion:

En conclusion, ce projet a permis de réaliser une application complète de gestion d'une pharmacie en Java avec une base de données SQLite, en utilisant uniquement JDBC. L'application répond aux besoins principaux : authentification des utilisateurs (pharmacien/gestionnaire), gestion des médicaments et du stock, enregistrement et consultation des ventes, gestion des commandes de réapprovisionnement, ainsi que le suivi du stock critique et de l'historique des mouvements de stock.

Ce travail a également permis de mettre en pratique une architecture en couches (DAO/Service/App), ce qui rend le code plus clair, mieux organisé et plus facile à maintenir. Des améliorations futures peuvent inclure une interface graphique plus avancée, des rapports plus détaillés et une gestion plus complète des notifications.

Exemples d'opérations de gestion des médicaments

8. Conclusion

Ce projet a abouti à la réalisation d'une application de gestion de pharmacie en Java, utilisant SQLite comme système de stockage et JDBC pour l'accès aux données. Les fonctionnalités décrites couvrent l'authentification des utilisateurs, la gestion des médicaments et du stock, l'enregistrement et la consultation des ventes, la gestion des commandes de réapprovisionnement, le suivi du stock critique et l'historique des mouvements de stock.

La séparation en couches DAO, Service et App contribue à organiser le code et à faciliter sa maintenance. Parmi les évolutions possibles figurent l'amélioration de l'interface graphique, l'ajout de rapports plus détaillés et le développement d'un système de notifications plus complet.

Auteur : Youssef Jbeniani